Reimbursement Management System

Database design and the justification of every design decision

DBMS Laboratory — group project

Version 3 · 28 October 2026

MIS number	Name
612415001	Abdurrahman Shaikh
612415057	Dhruv Agarwal
612415044	Mrudula Dagale
612415093	Makarand S Kulkarni

Scope: this document covers the database (ER diagram, relational schemas, functional dependencies, normalization and integrity constraints). The SQL scripts in db/ implement it.

Contents

1 About this document 3

2 Problem statement 3

3 Objectives 3

4 Functional requirements 4

5 Changes from the original ER diagram 4

6 ER diagram 5

6.1 How to read the arrows and lines 5

6.2 Every relationship and arrow 5

6.3 Every specialization 6

6.4 Entity sets 8

6.5 Attributes 9

6.6 Relationship sets 11

6.7 Specialization, aggregation and the weak entity set 13

7 Relational schemas obtained from the ER diagram 14

7.1 Reduction decisions 15

8 Schema diagram and column definitions 17

8.1 Columns, types and nulls 18

9 Functional dependencies of each table 20

9.1 Dependencies we deliberately do not assert 20

10 Normalization up to 3NF 21

10.1 The definitions we test against 21

10.2 Table by table 21

10.3 Normalizing the flat claim form 22

10.4 The decomposition is lossless and dependency-preserving 23

11 Integrity constraints 23

11.1 Constraint catalogue 23

11.2 What happens on delete 24

11.3 Claim status workflow 25

11.4 Constraint decisions 25

12 Open questions for the team 26

Appendix A Assignment checklist 27

Appendix B Questions to expect in the viva 27

1 About this document

The design follows the team's original ER diagram (references/reimbursement-management.pdf) and the seven changes accepted in the project plan. Every decision has an ID, used across the document and in the viva:

Prefix	Covers	Section
E	Entity sets	6.4
A	Attributes	6.5
R	Relationship sets	6.6
S, G, W	Specialization, aggregation, weak entity set	6.7
M	Reduction of the ER diagram to tables	7
N	Functional dependencies we deliberately do not assert	9.1
K	Constraint strategy	11
C, T, A	Individual constraints: keys and CHECK, cross-table rules checked by the application, other application rules	11.1
Q	Open questions for the team	12

Each decision is a small table: what we decided, why, the alternative we rejected and what it costs. Textbook references (§) are to Silberschatz, Korth & Sudarshan, *Database System Concepts*, 7th edition.

The assignment asks for seven report items. Appendix A shows where each one is in this document.

2 Problem statement

Employees spend their own money on company work: travel to a client, a hotel during a business trip, a medical bill covered by company policy. Today these claims travel as paper forms and spreadsheets. Bills get lost. Nobody can see what state a claim is in. A manager can approve an amount beyond their authority, or even their own claim. Finance cannot easily answer "how much did each department spend on travel this quarter?".

We need a database-backed system in which an employee files a typed claim (travel, medical or business) made up of individual bills. A manager other than the claimant approves or rejects it within their approval limit and records a reason. The claim is then marked paid into the claimant's registered bank account. Every claim and decision must be recorded consistently and kept as a financial record.

3 Objectives

1. Store employees, departments, reporting lines and roles (applicant, manager) without redundancy.
2. Record each claim with its type-specific details and its individual bills, with receipts.
3. Enforce the approval rules: no self-approval and a reason for every rejection as CHECK constraints in the database; approval within the manager's limit and only valid status changes in the application, checked in the form and again in the API.
4. Keep every table in at least 3NF (in fact BCNF), so that there are no update, insertion or deletion anomalies.
5. Support reports such as totals by department, type, category and month.

6. Document every design decision with its justification and the alternatives considered.

4 Functional requirements

The requirements below are the ones the database has to support. The last column shows the tables each one uses.

#	Requirement	Tables
FR1	An employee logs in with email and password.	employee
FR2	The system knows each user's roles: applicant, manager, both or neither.	applicant, manager
FR3	Departments, employees, their phone numbers, supervisors and roles can be added and edited.	department, employee, employee_phone, applicant, manager
FR4	An applicant submits a claim of exactly one type (travel, medical or business) with its type-specific details and one or more expense items, each with a category, amount, date and optional receipt.	reimbursement, travel_/medical_ /business_reimbursement, expense_item
FR5	An applicant sees their claims with total, status, decision date and remarks.	reimbursement, expense_item (view reimbursement_summary)
FR6	A manager sees submitted claims, other than their own, whose total is within their approval limit.	reimbursement, manager, expense_item
FR7	A manager approves or rejects a claim; a rejection requires remarks; the decision date is recorded.	reimbursement
FR8	An approved claim is marked paid.	reimbursement
FR9	Reports: totals by department, claim type, category and month; claims by status.	all, through joins and GROUP BY

5 Changes from the original ER diagram

All seven changes proposed in the project plan were accepted. Nothing else about the original structure changed: the same entity sets, the same overlapping and disjoint specializations, the same aggregation and the same weak entity.

#	Change	Why	Decision
1	approval_date and remarks added to APPROVES	A rejection needs a reason; the decision date must be recorded	A9
2	Claim specialization marked disjoint and total; claim_type added	Each claim is exactly one type	S2, A6
3	Status values made explicit: SUBMITTED, APPROVED, REJECTED, PAID	Defines the workflow; enables a CHECK	A5
4	DEPARTMENT made an entity set (dept_id, dept_name, location)	Removes a transitive dependency	E1, R1
5	Receipt stored as a file path	Files stay on disk; the database holds the path	A7
6	Category added to EXPENSE_ITEM	Grouping in reports	A8

#	Change	Why	Decision
7	password_hash added to EMPLOYEE	Users must log in	A4

Small clarifications that follow from these changes, all explained in Section 6: attribute names are written in snake_case (M4); Distance is named distance_km to state its unit; total_amount is shown as a derived attribute; and the original Department attribute is replaced by the WORKS_IN relationship.

6 ER diagram

The diagram on the next page uses the notation of textbook Chapter 6. Attributes are listed inside the entity rectangle (the textbook's own style; the original diagram used ovals, §6.10.1). The key at the bottom explains every symbol. The tables below then list every relationship and every arrow, with the reason for each.

6.1 How to read the arrows and lines

Symbol	Meaning
Arrow (→) on a line	The "one" side. Each entity at the other end relates to at most one entity here (§6.4).
Plain line	The "many" side.
Double line	Total participation: every entity of that set takes part (§6.4).
l..h on a line	Minimum..maximum number of relationships each entity of that set takes part in. 1..1 = exactly one; 0..* = any number.
Role label (supervisor, subordinate)	Which end is which, when an entity set relates to itself.
Double rectangle; dashed underline	Weak entity set; its discriminator (§6.5.3).
Double diamond	Identifying relationship of a weak entity set.
Hollow arrowhead	ISA: points to the superclass. Separate arrows = overlapping; one shared arrow = disjoint; "total" keyword with a dashed line = total (§6.8.4).
Box around a relationship	Aggregation: the relationship and its entity sets treated as one entity set (§6.8.5).
{ phone }; total_amount ( )	Multivalued attribute; derived attribute (§6.3).
Small box with a dashed link to a diamond	Descriptive attributes of that relationship.

6.2 Every relationship and arrow

Relationship	Between	Arrow points to	Lines (participation)	l..h	Decision
WORKS_IN	EMPLOYEE, DEPARTMENT	DEPARTMENT	EMPLOYEE double (total); DEPARTMENT single	EMPLOYEE 1..1; DEPARTMENT 0..*	R1

Relationship	Between	Arrow points to	Lines (participation)	l..h	Decision
SUPERVISES	EMPLOYEE (supervisor), EMPLOYEE (subordinate)	supervisor role	both single (partial)	subordinate 0..1; supervisor 0..*	R2
SUBMITS	APPLICANT, REIMBURSEMENT	APPLICANT	REIMBURSEMENT double (total); APPLICANT single	REIMBURSEMENT 1..1; APPLICANT 0..*	R3
APPROVES	MANAGER, CLAIM_PROCESS (aggregation)	MANAGER	both single (partial)	CLAIM_PROCESS 0..1; MANAGER 0..*	R4, G1
CONTAINS (identifying)	REIMBURSEMENT, EXPENSE_ITEM	REIMBURSEMENT	both double (total)	EXPENSE_ITEM 1..1; REIMBURSEMENT 1..*	R5, W1

6.3 Every specialization

Superclass	Subclasses	Arrows	Disjoint / overlapping	Total / partial	Decision
EMPLOYEE	APPLICANT, MANAGER	two separate hollow arrows	overlapping	partial (no keyword)	S1
REIMBURSEMENT	TRAVEL_, MEDICAL_, BUSINESS_REIMBURSEMENT	one shared hollow arrow	disjoint	total (keyword "total")	S2

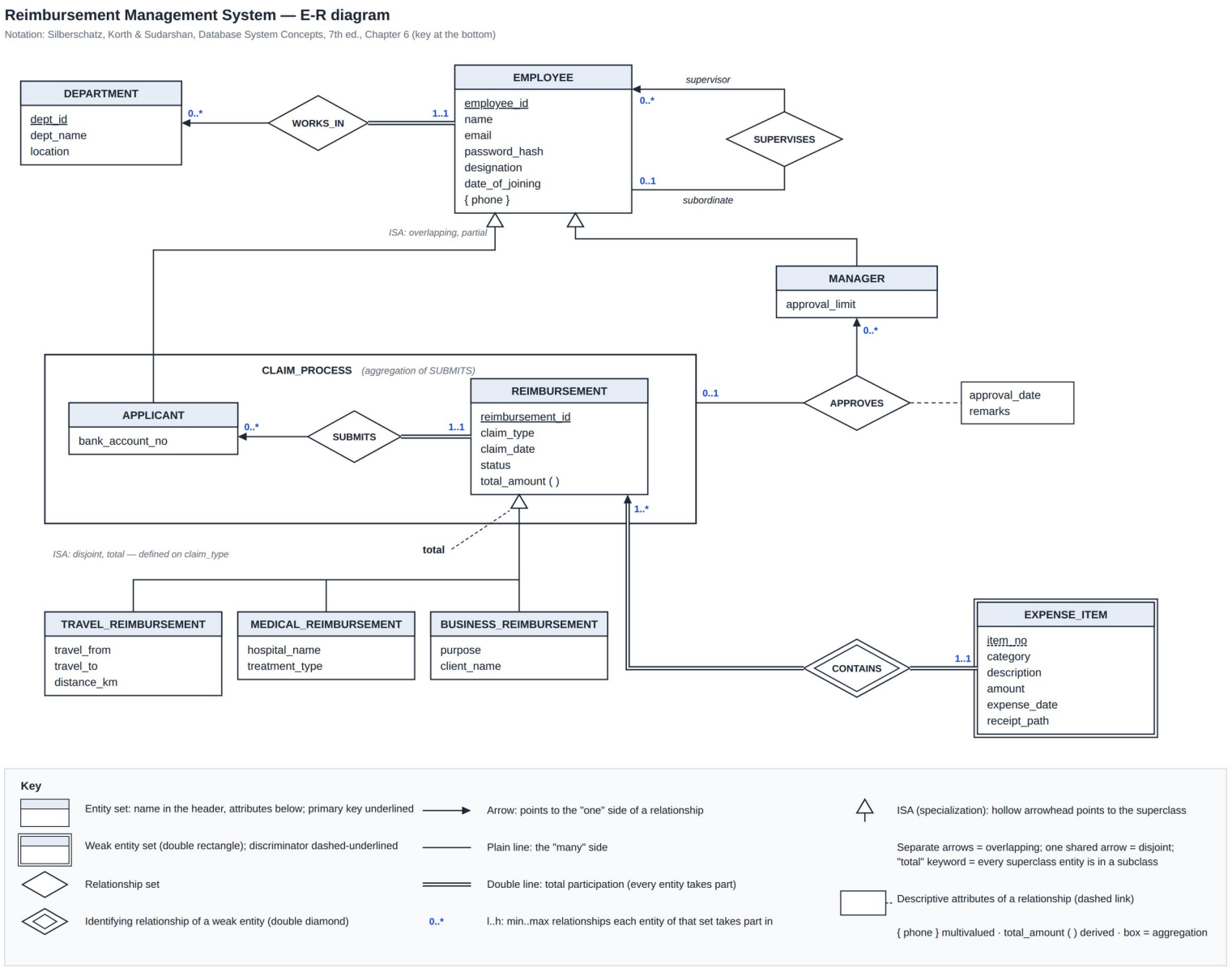


Figure 1. ER diagram of the Reimbursement Management System, with the key to the notation.

6.4 Entity sets

E1 DEPARTMENT is an entity set, not an attribute (approved change 4)

Decision	Model DEPARTMENT (dept_id, dept_name, location) and link EMPLOYEE to it through WORKS_IN.
Why	A department has a fact of its own (its location) and is shared by many employees. Textbook §6.9.2: when something has attributes of its own, model it as an entity set. As a text attribute of EMPLOYEE, the department name repeats in every employee row, and a typo silently creates a new department. Adding location to EMPLOYEE would give employee_id → department → location, a transitive dependency that breaks 3NF (§7.3.2).
Alternative rejected	Department as a text attribute of EMPLOYEE (the original diagram).
Cost	One more table, and a join wherever the department name is shown.

E2 EMPLOYEE is the superclass of the two roles

Decision	EMPLOYEE holds everything every person in the company has: id, name, email, password hash, designation, date of joining, phones, department, supervisor. APPLICANT and MANAGER are specializations (S1).
Why	These attributes do not depend on the role. Keeping them once in EMPLOYEE means a manager who also files claims is stored once, not twice (§6.8.3, attribute inheritance).
Alternative rejected	Separate APPLICANT and MANAGER entity sets, each with name, email and so on. A person with both roles would be stored twice and could drift out of sync.

E3 REIMBURSEMENT (the claim) and EXPENSE_ITEM (its lines) are separate

Decision	REIMBURSEMENT is the claim header: who, when, which type, what status. Each bill is an EXPENSE_ITEM inside it.
Why	A claim has many bills (a trip has a fare, a hotel and meals). Putting amount, date and receipt on the claim would allow one bill per claim, or force repeating groups, which violate 1NF (§7.8).
Alternative rejected	One amount per claim. Simpler, but a real trip would need several claims and the per-bill audit trail would be lost.

E4 No separate ADMIN or FINANCE entity set in this version

Decision	The approved plan has no admin entity, so the design has none. Master data (departments, employees, roles) and the "mark as paid" step are application functions, not modelled roles. In the application, managers perform them; nobody may mark their own claim paid.
Why	We keep to the approved plan. An ADMIN role would only be justified if it had its own attributes or relationships (§6.8.1), for example "PAID BY". The approved plan does not record who pays.
Open question	Q2: if the team wants to record who paid and when, we add ADMIN as a third subclass of EMPLOYEE and a PAYS relationship. The rest of the schema does not change.

6.5 Attributes

A1 Surrogate integer keys for the strong entity sets

Decision	dept_id, employee_id and reimbursement_id are system-generated integers (AUTO_INCREMENT).
Why	The natural candidates can change: a person changes their email, a department is renamed. A primary key must never change, because foreign keys in other tables copy it (§6.5). An integer key is small, so every foreign key and index built on it is small too. The natural keys are still enforced as candidate keys with UNIQUE (email, dept_name, bank_account_no), so no duplicates can sneak in.
Alternative rejected	email as the primary key of employee. It would be copied into reimbursement, employee_phone and so on, and an email change would have to cascade through all of them.

A2 Phone is multivalued

Decision	{ phone } is a multivalued attribute of EMPLOYEE. It becomes the table employee_phone (employee_id, phone).
Why	An employee can have any number of phone numbers. Columns phone1, phone2 cap the number and create NULLs. A comma-separated list in one column is not atomic and breaks 1NF (§7.8). A phone number has no attributes of its own, so a PHONE entity set would add nothing (§6.9.2).
Textbook	§6.3 (multivalued attributes), §6.7.2 (their reduction to a separate schema)

A3 total_amount is derived, not stored

Decision	total_amount ( ) is a derived attribute of REIMBURSEMENT: the sum of its expense item amounts. It is computed by the view reimbursement_summary and not stored in any table.
Why	Storing it would create a second copy of the same fact. When an item is added or changed, the stored total could disagree with the items: an update anomaly (§7.1). Computing it is cheap, because a claim has only a few items.
Alternative rejected	A stored total kept up to date on every item change. It is faster to read, but it is redundant and one more thing that can go wrong.
Textbook	§6.3 (derived attributes are not stored), §4.2 (views)

A4 password_hash on EMPLOYEE (approved change 7)

Decision	EMPLOYEE gets password_hash CHAR(60). Only a bcrypt hash is stored, never the password.
Why	Every employee logs in, whatever their role, so the attribute belongs to the superclass. The email is the login name; it is already a candidate key. A bcrypt hash is always 60 characters, hence CHAR(60).

A5 status is stored, with four explicit values (approved change 3)

Decision	status ∈ {SUBMITTED, APPROVED, REJECTED, PAID}. The allowed moves are in the status diagram (Section 11.3).
Why	A fixed list makes the workflow precise and lets the database reject anything else with a CHECK (C5). status cannot be derived: approved_by tells us whether a decision was made, but not whether it was approval or rejection, nor whether the money was paid.

A6 claim_type on REIMBURSEMENT (approved change 2)

Decision	claim_type ∈ {TRAVEL, MEDICAL, BUSINESS} is an attribute of REIMBURSEMENT. The claim specialization (S2) is defined on its value.
Why	Without it, the type of a claim can only be found by probing three tables. With it, a single row says which subclass table must hold the claim's details. It makes disjointness checkable: the application (rule T3) writes the details row only to the table that matches claim_type, so a MEDICAL claim never gets a TRAVEL row. It is a fact about the claim, determined by reimbursement_id alone, so it causes no normalization problem (Section 10).

A7 Receipt stored as a file path (approved change 5)

Decision	EXPENSE_ITEM.receipt_path VARCHAR(255) holds the path of the uploaded file. The file itself lives on disk. The column may be NULL.
Why	Images and PDFs in the database (BLOBs) make tables and backups large and slow, while the database gains nothing from them: it never searches inside a receipt. NULL is allowed because small expenses (an auto-rickshaw fare) often have no bill. The manager sees that the receipt is missing and decides.
Cost	The database cannot guarantee the file exists. The application writes the file first and stores the path in the same request.

A8 Category on EXPENSE_ITEM (approved change 6)

Decision	category ∈ {FOOD, LODGING, FARE, FUEL, LOCAL_TRANSPORT, MEDICAL, SUPPLIES, OTHER}.
Why	Reports such as "spend on lodging per department" need a fixed set to group by. Free text would produce "Hotel", "hotel" and "Lodging" as three groups.
Alternative rejected	A category lookup table. See K4: a lookup table pays off only when the values have attributes of their own or change often.

A9 approval_date and remarks belong to APPROVES (approved change 1)

Decision	The date of the manager's decision and the manager's remarks are descriptive attributes of the APPROVES relationship, not of REIMBURSEMENT.
Why	They describe the decision, not the claim. They exist only when an APPROVES relationship exists, which is exactly the rule for placing an attribute on a relationship (§6.2.2, §6.9). A rejection needs a reason (rule C9), and the record must show when the decision was taken.

Note	APPROVES records the decision whether the manager approves or rejects; status says which. approval_date is therefore "the date of the decision".

A10 approval_limit belongs to the individual MANAGER

Decision	Each manager has an approval_limit. A manager may approve a claim only if its total is within that limit (rule T1, checked by the application).
Why	Limits are given to people, not to designations: two "Team Lead"s can have different limits. So we do not assert designation → approval_limit, and the limit is stored per manager.

A11 bank_account_no belongs to APPLICANT only

Decision	The payout account is an attribute of the APPLICANT subclass and is UNIQUE.
Why	Only employees who claim money need a payout account. Placing it on EMPLOYEE would leave it NULL for everyone else. UNIQUE: two employees must not route their reimbursements into the same account, a common fraud pattern. It also makes bank_account_no a second candidate key of applicant.
Open question	Q4: Indian account numbers are unique only within a bank. If the team adds ifsc_code, the candidate key becomes (ifsc_code, bank_account_no). The approved plan has only the account number, so we keep that.

A12 Simple, atomic attributes elsewhere

Decision	name is one attribute (not first/middle/last). Places (travel_from, travel_to, hospital_name) are plain text.
Why	No requirement searches or sorts by a part of a name or an address, so splitting them adds columns and gains nothing. Each value is still atomic for our purposes, so 1NF holds (§7.8: atomicity depends on how the data is used).

6.6 Relationship sets

R1 WORKS_IN: EMPLOYEE many-to-one DEPARTMENT

Mapping	Many-to-one: arrow points at DEPARTMENT. An employee works in exactly one department; a department has any number of employees.
Participation	EMPLOYEE total (double line, 1..1): every employee must belong to a department. DEPARTMENT partial (0..*): a newly created department may have nobody in it yet.
Result	employee.dept_id NOT NULL, foreign key to department.

R2 SUPERVISES: recursive one-to-many on EMPLOYEE

Mapping	One supervisor, many subordinates: the arrow points at the supervisor role. Role labels "supervisor" and "subordinate" are needed because both ends are EMPLOYEE (§6.2.2).
Participation	Both partial. Subordinate side 0..1: the head of the company has no supervisor. Supervisor side 0..*: most employees supervise nobody.

Why on EMPLOYEE	Reporting lines exist for everyone, not only managers. A team lead may supervise people without having any approval authority.
Result	employee.supervisor_id NULL, foreign key to employee. A self-reference and longer cycles are refused by the application (rule T4).

R3 SUBMITS: APPLICANT one-to-many REIMBURSEMENT

Mapping	Each claim is submitted by exactly one applicant; an applicant submits any number of claims. Arrow points at APPLICANT.
Participation	REIMBURSEMENT total (1..1): a claim without a claimant is meaningless. APPLICANT partial (0..*): a new applicant has no claims yet.
Result	reimbursement.applicant_id NOT NULL, foreign key to applicant (not to employee). The database itself guarantees that only registered applicants, who have a payout account, can own claims.

R4 APPROVES: MANAGER one-to-many CLAIM_PROCESS

Mapping	Each claim is decided by at most one manager; a manager decides any number of claims. Arrow points at MANAGER.
Participation	Both partial, as in the original diagram. CLAIM_PROCESS 0..1: a claim that is still SUBMITTED has no decision. MANAGER 0..*: a new manager has decided nothing.
Single level	One decision per claim. Multi-level approval would need a many-to-many relationship and a separate table; the approved plan leaves it as an open question (Q3).
Result	reimbursement.approved_by, approval_date, remarks, all nullable (see M2).

R5 CONTAINS: identifying relationship of EXPENSE_ITEM

Mapping	One claim, many items. Arrow points at REIMBURSEMENT. Double diamond: it is the identifying relationship of the weak entity set (W1).
Participation	EXPENSE_ITEM total (1..1): an item cannot exist without its claim; this is always so for a weak entity set (§6.5.3). REIMBURSEMENT total too (1..*): a claim with no items has nothing to reimburse.
Result	No table of its own (§6.7.5); expense_item carries reimbursement_id as part of its key.

R6 The approver does not have to be the applicant's supervisor

Decision	SUPERVISES and APPROVES stay independent, as in the original diagram. Any manager other than the applicant may decide a claim, within their limit.
Why	The applicant's supervisor may not have a high enough limit for a large claim; another manager must then be able to decide it. A reporting line and an approval authority are different facts, so they are different relationships.
Note	The plan marked this question as settled but did not record the answer. This is our assumption; see Q1. If the team wants "supervisor only", it is one extra check in rule T1, with no change to the tables.

R7 All relationships are binary

Decision	No ternary relationships. The one place where three entity sets meet (manager, applicant, claim) is modelled with aggregation (G1).
Why	A ternary relationship APPROVES (MANAGER, APPLICANT, REIMBURSEMENT) would record the applicant twice, once in SUBMITS and once in APPROVES, and nothing would stop them from disagreeing (§6.9.4, §6.8.5).

6.7 Specialization, aggregation and the weak entity set

S1 EMPLOYEE → {APPLICANT, MANAGER}: overlapping and partial

Overlapping	Two separate hollow arrows. A manager may also file claims, as the original notes say, so one employee can be in both subclasses.
Partial	No "total" keyword. Some employees have neither role, for example a new joiner who has not registered a bank account yet.
Why subclasses	Each role has its own attribute (bank_account_no, approval_limit) and its own relationship (SUBMITS, APPROVES). A foreign key must be able to point at "a manager", not just "an employee": reimbursement.approved_by references manager, so a non-manager can never appear as the approver.
Alternative rejected	A role column on employee. It cannot hold role-specific attributes without NULLs, cannot express overlap without a list, and cannot be the target of a foreign key.

S2 REIMBURSEMENT → {TRAVEL, MEDICAL, BUSINESS}: disjoint and total (approved change 2)

Disjoint	One shared hollow arrow. A claim has exactly one purpose; a business trip with a medical visit is filed as two claims.
Total	The keyword "total". Every claim is one of the three types; "untyped" claims do not exist.
Enforcement	claim_type (A6) states the type. Disjointness: rule T3, checked by the application. Totality: the claim row, its subclass row and its items are inserted in one transaction (A1 in Section 11). MySQL has no assertions or deferred constraints, so the database alone cannot require a subclass row (§4.4.7, §4.4.8).

S3 Both specializations are reduced with the textbook's method 1

Decision	One table for the superclass, and one per subclass holding the subclass attributes plus the superclass key, which is both primary key and foreign key (§6.8.6.1, method 1).
Why not method 2 for EMPLOYEE	Method 2 (no superclass table) is allowed only for disjoint, total specializations. EMPLOYEE's is overlapping (name and email would be stored twice) and partial (employees with no role would have no table).
Why not method 2 for REIMBURSEMENT	It is disjoint and total, so method 2 is allowed, but CONTAINS and APPROVES then have no single table to reference. expense_item.reimbursement_id would have to point at one of three tables, which a foreign key cannot do. This is exactly the drawback the textbook gives.

G1 Aggregation CLAIM_PROCESS

Decision	APPLICANT – SUBMITS – REIMBURSEMENT is treated as one higher-level entity, CLAIM_PROCESS, and MANAGER APPROVES that, as in the original diagram.
Why	What a manager decides on is a submission: this applicant claims this money. The rule "a manager may not approve their own claim" compares the approver with the submitter of the same CLAIM_PROCESS. A ternary relationship would duplicate the applicant (R7). This is the case the textbook uses to motivate aggregation (§6.8.5).
Viva note	Because SUBMITS is many-to-one, the key of CLAIM_PROCESS is just reimbursement_id (§6.8.6.2). A plain binary MANAGER–REIMBURSEMENT relationship would therefore give the same tables. We keep the aggregation because it says what is decided; be ready to state this.

W1 EXPENSE_ITEM is a weak entity set

Decision	EXPENSE_ITEM has no key of its own. It is identified by its claim plus the discriminator item_no (1, 2, 3 … within each claim). Double rectangle, dashed underline on item_no.
Why	An item has no meaning outside its claim: "item 2" means "item 2 of claim 41". That is existence dependence, the definition of a weak entity set (§6.5.3). Deleting a claim must delete its items (ON DELETE CASCADE).
Alternative rejected	A strong entity with a surrogate item_id. It works technically, but it hides the dependence on the claim, and the textbook construct is what the original diagram shows.

7 Relational schemas obtained from the ER diagram

The ER diagram reduces to 10 tables and 1 view. Primary keys are underlined. The last column gives the reduction rule that produces each table.

Table	Schema	Comes from
department	department (dept_id, dept_name, location)	Entity set DEPARTMENT (§6.7.1)
employee	employee (employee_id, name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id)	Entity set EMPLOYEE, plus WORKS_IN and SUPERVISES folded in (§6.7.6)
employee_phone	employee_phone (employee_id, phone)	Multivalued attribute { phone } (§6.7.2)
applicant	applicant (employee_id, bank_account_no)	Subclass of EMPLOYEE (§6.8.6.1, method 1)
manager	manager (employee_id, approval_limit)	Subclass of EMPLOYEE (§6.8.6.1, method 1)
reimbursement	reimbursement (reimbursement_id, applicant_id, claim_type, claim_date, status, approved_by, approval_date, remarks)	Entity set REIMBURSEMENT, plus SUBMITS and APPROVES folded in (§6.7.6, §6.8.6.2)
travel_reimbursement	travel_reimbursement (reimbursement_id, travel_from, travel_to, distance_km)	Subclass of REIMBURSEMENT (method 1)
medical_reimbursement	medical_reimbursement (reimbursement_id, hospital_name, treatment_type)	Subclass of REIMBURSEMENT (method 1)
business_reimbursement	business_reimbursement (reimbursement_id, purpose, client_name)	Subclass of REIMBURSEMENT (method 1)

Table	Schema	Comes from
expense_item	expense_item (reimbursement_id, item_no, category, description, amount, expense_date, receipt_path)	Weak entity set EXPENSE_ITEM; CONTAINS needs no table (§6.7.3, §6.7.5)

View reimbursement_summary (reimbursement_id, applicant_id, claim_type, claim_date, status, approved_by, approval_date, remarks, total_amount, item_count): the derived attribute total_amount (A3), computed as SUM(expense_item.amount) grouped by claim.

What has no table, and why:

- WORKS_IN, SUPERVISES, SUBMITS, APPROVES: many-to-one, folded into the "many" side (M1, M2).
- CONTAINS: its schema is contained in expense_item (M3).
- CLAIM_PROCESS: an aggregation uses the table of its defining relationship (M3).

7.1 Reduction decisions

M1 Many-to-one relationships are folded into the "many" side

Decision	WORKS_IN, SUPERVISES, SUBMITS and APPROVES get no tables. Each becomes a foreign key column (plus descriptive attributes) in the table on the "many" side.
Why	Textbook §6.7.6 (combination of schemas). With total participation the column is NOT NULL (dept_id, applicant_id). With partial participation it is nullable; the textbook allows this explicitly (supervisor_id, approved_by).

M2 APPROVES is folded into reimbursement, not given its own table

Decision	reimbursement gets approved_by, approval_date and remarks, all nullable.
Alternative	A separate table approval (reimbursement_id PK and FK, manager_id, approval_date, remarks). It has no NULLs, because a row exists only when a decision exists.
Why we fold	The rules that link the decision to the claim then become single-row CHECK constraints: "SUBMITTED ⇔ no approver" (C8), "REJECTED ⇒ remarks" (C9), "approver ≠ applicant" (C10), "approval_date ≥ claim_date" (C11). With a separate table, every one of these crosses two tables and could no longer be a CHECK. Folding also saves a join on every claim screen.
Cost	Three nullable columns that are NULL while a claim is pending. The textbook accepts this for partial participation (§6.7.6).

M3 No tables for CONTAINS or for the aggregation

Decision	CONTAINS has no table: its schema would be (reimbursement_id, item_no), already inside expense_item (§6.7.5, redundancy of schemas). CLAIM_PROCESS has no table either: the aggregation uses the table of its defining relationship (§6.8.6.2), which is reimbursement.

M4 Naming conventions

Decision	Tables and columns are lower-case snake_case and singular (employee, not employees). A foreign key is named after the role it plays, not the table it points to: applicant_id, approved_by, supervisor_id.
Why	The role name says what the reference means. "approved_by → manager" reads correctly; "manager_id" would not say whether this manager approved, supervised or submitted. Textbook §7.9.2.

8 Schema diagram and column definitions

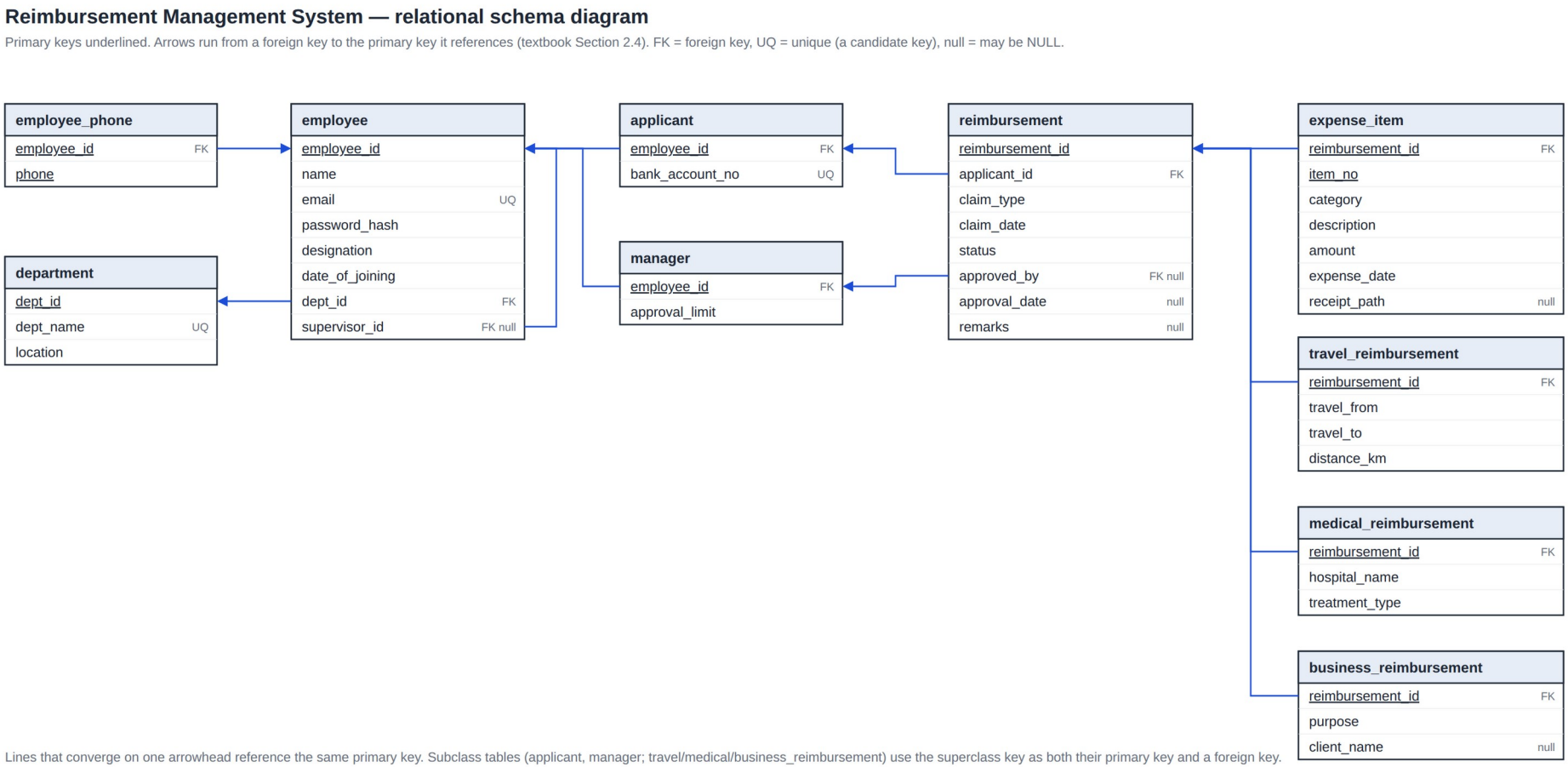


Figure 2. Relational schema diagram. Each arrow goes from a foreign key to the primary key it references.

8.1 Columns, types and nulls

"Null" says whether the column may be NULL. Every nullable column comes from partial participation or an optional value, and the reason is given beside it.

department

Column	Type	Null	Key / constraint / note
dept_id	INT	no	PK, auto-generated
dept_name	VARCHAR(60)	no	UNIQUE (candidate key)
location	VARCHAR(60)	no	

employee

Column	Type	Null	Key / constraint / note
employee_id	INT	no	PK, auto-generated
name	VARCHAR(80)	no	
email	VARCHAR(100)	no	UNIQUE (candidate key), login name; C14
password_hash	CHAR(60)	no	bcrypt hash (A4)
designation	VARCHAR(50)	no	
date_of_joining	DATE	no	
dept_id	INT	no	FK → department, RESTRICT (WORKS_IN, total)
supervisor_id	INT	yes	FK → employee, RESTRICT (SUPERVISES, partial); T4 (application)

employee_phone

Column	Type	Null	Key / constraint / note
employee_id	INT	no	PK part; FK → employee, CASCADE
phone	VARCHAR(15)	no	PK part; text, to keep "+91" and leading zeros

applicant

Column	Type	Null	Key / constraint / note
employee_id	INT	no	PK; FK → employee, CASCADE
bank_account_no	VARCHAR(18)	no	UNIQUE (candidate key); digits as text

manager

Column	Type	Null	Key / constraint / note
employee_id	INT	no	PK; FK → employee, CASCADE
approval_limit	DECIMAL(12,2)	no	C12: > 0

reimbursement

Column	Type	Null	Key / constraint / note
reimbursement_id	INT	no	PK, auto-generated
applicant_id	INT	no	FK → applicant, RESTRICT (SUBMITS, total)
claim_type	VARCHAR(10)	no	C6
claim_date	DATE	no	default: today
status	VARCHAR(10)	no	C5; default SUBMITTED; T1 (application)
approved_by	INT	yes	FK → manager, RESTRICT (APPROVES); C8, C10
approval_date	DATE	yes	C8, C11
remarks	VARCHAR(255)	yes	C9

travel_reimbursement

Column	Type	Null	Key / constraint / note
reimbursement_id	INT	no	PK; FK → reimbursement, CASCADE; T3
travel_from	VARCHAR(100)	no	
travel_to	VARCHAR(100)	no	
distance_km	DECIMAL(7,1)	no	C12: > 0

medical_reimbursement

Column	Type	Null	Key / constraint / note
reimbursement_id	INT	no	PK; FK → reimbursement, CASCADE; T3
hospital_name	VARCHAR(100)	no	
treatment_type	VARCHAR(60)	no	

business_reimbursement

Column	Type	Null	Key / constraint / note
reimbursement_id	INT	no	PK; FK → reimbursement, CASCADE; T3
purpose	VARCHAR(200)	no	
client_name	VARCHAR(100)	yes	internal work has no client

expense_item

Column	Type	Null	Key / constraint / note
reimbursement_id	INT	no	PK part; FK → reimbursement, CASCADE

Column	Type	Null	Key / constraint / note
item_no	SMALLINT	no	PK part; discriminator 1, 2, 3 … per claim
category	VARCHAR(20)	no	C7
description	VARCHAR(200)	no	
amount	DECIMAL(10,2)	no	C12: > 0
expense_date	DATE	no	T2 (application): not after claim_date
receipt_path	VARCHAR(255)	yes	path of the uploaded file (A7)

Identifiers that are digits (bank account, phone) are text, not numbers: they have leading zeros or a "+", and nobody does arithmetic on them. Dates are DATE, because no rule needs the time of day.

9 Functional dependencies of each table

For each table: the non-trivial functional dependencies that must hold, its candidate keys, and why it is in BCNF. Multi-line cells list one FD per line. Every FD listed is enforced: an FD whose left side is the primary key by the PRIMARY KEY, one whose left side is another candidate key by UNIQUE.

Table	Non-trivial FDs (a canonical cover)	Candidate keys	BCNF because
department	dept_id → dept_name, location dept_name → dept_id	{dept_id}, {dept_name}	Both left sides are candidate keys.
employee	employee_id → name, email, password_hash, designation, date_of_joining, dept_id, supervisor_id email → employee_id	{employee_id}, {email}	Both left sides are candidate keys.
employee_phone	none except trivial ones	{employee_id, phone}	All-key relation: no non-trivial FD can exist.
applicant	employee_id → bank_account_no bank_account_no → employee_id	{employee_id}, {bank_account_no}	Both left sides are candidate keys.
manager	employee_id → approval_limit	{employee_id}	Left side is the key.
reimbursement	reimbursement_id → applicant_id, claim_type, claim_date, status, approved_by, approval_date, remarks	{reimbursement_id}	Left side is the key.
travel_reimbursement	reimbursement_id → travel_from, travel_to, distance_km	{reimbursement_id}	Left side is the key. See N1 for (travel_from, travel_to) → distance_km.
medical_reimbursement	reimbursement_id → hospital_name, treatment_type	{reimbursement_id}	Left side is the key.
business_reimbursement	reimbursement_id → purpose, client_name	{reimbursement_id}	Left side is the key.
expense_item	(reimbursement_id, item_no) → category, description, amount, expense_date, receipt_path	{reimbursement_id, item_no}	Left side is the key.

9.1 Dependencies we deliberately do not assert

A strict reading of the data might suggest more FDs. These do *not* hold in our enterprise, and the reason matters, because asserting any of them would change the design.

N1 FDs that do not hold in our enterprise

(travel_from, travel_to) → distance_km	distance_km is the distance of this trip as claimed (the route actually taken, detours, a round trip), not a property of the two places. Two trips between the same places can differ. If the company paid by a fixed distance per city pair, this FD would hold and travel_reimbursement would not be in 3NF; we would then decompose it into route (travel_from, travel_to, distance_km).
designation → approval_limit	Limits are given to individual managers (A10).
dept_id → supervisor_id	A department can have several supervisors (team leads).
phone → employee_id	A desk phone can be shared, so a number does not identify a person. It makes no difference anyway: employee_phone is all-key.
hospital_name → treatment_type	The same hospital treats many conditions.
approved_by → anything in reimbursement	A manager decides many claims with different dates and remarks.
applicant_id → claim_type (or any claim attribute)	An applicant files many claims of different types.

10 Normalization up to 3NF

Result: every table is in BCNF, and therefore in 3NF (§7.3: BCNF implies 3NF). This section proves it per table, then shows that normalizing a flat "claim form" relation from first principles produces the same tables as the ER reduction.

10.1 The definitions we test against

- **BCNF (§7.3.1):** for every non-trivial FD $\alpha \rightarrow \beta$ in F+, α is a superkey.
- **3NF (§7.3.2):** for every FD $\alpha \rightarrow \beta$ in F+, either it is trivial, or α is a superkey, or every attribute in $\beta - \alpha$ is contained in some candidate key.
- **1NF (§7.8):** all attribute domains are atomic.
- **2NF** (Exercise 7.19; the textbook calls it of historical interest): no non-prime attribute depends on a proper subset of a candidate key.

10.2 Table by table

In every table of Section 9, the left side of every non-trivial FD is a candidate key, hence a superkey. That is the BCNF condition, so all 10 tables are in BCNF, and so in 3NF. Two cases need a sentence more:

- **Tables with two candidate keys** (department, employee, applicant): the extra FDs dept_name → dept_id, email → employee_id and bank_account_no → employee_id also have a candidate key on the left, so they are allowed.
- **employee_phone** is all-key: its only key is (employee_id, phone) and no non-trivial FD holds. It is also in 4NF: the multivalued dependency employee_id ↠ phone has a superkey on its left once phone is in its own table (§7.6).

10.3 Normalizing the flat claim form

Suppose all claim data were kept in one relation, the way a paper form or a spreadsheet does (subclass details left out for brevity; they follow the same pattern):

```
claim_form (reimbursement_id, claim_date, claim_type, status,
            applicant_id, name, email, bank_account_no, dept_id, dept_name, location, phones,
            approved_by, approval_limit, approval_date, remarks,
            item_no, category, description, amount, expense_date, receipt_path)
```

Anomalies in this relation (§7.1):

- **Update:** an employee's email is repeated on every item of every claim; changing it in one row leaves the others wrong.
- **Insertion:** a new department cannot be stored until somebody in it files a claim.
- **Deletion:** deleting the only claim of a department also deletes the department's location.

Functional dependencies F (abbreviating reimbursement_id as rid):

```
F1  rid           → claim_date, claim_type, status, applicant_id, approved_by, approval_date,
remarks
F2  applicant_id  → name, email, bank_account_no, dept_id
F3  email         → applicant_id
F4  bank_account_no → applicant_id
F5  dept_id       → dept_name, location
F6  dept_name     → dept_id
F7  approved_by   → approval_limit
F8  rid, item_no  → category, description, amount, expense_date, receipt_path
```

F is already a canonical cover (§7.4.3): every right side is needed, no left side has an extraneous attribute, and no two FDs have the same left side.

Step 1: first normal form

phones holds a list, so the relation is not in 1NF. Repeating the row once per phone would make (rid, item_no, phone) the key and multiply every item by the number of phones. Instead we move it out: **employee_phone (employee_id, phone)**. What remains has atomic domains, and its only candidate key is (rid, item_no): the closure (rid, item_no)+ under F contains every attribute.

Step 2: second normal form

By F1, everything about the claim depends on rid alone, a proper subset of the key (a partial dependency). Split it off:

- **expense_item (rid, item_no, category, description, amount, expense_date, receipt_path)**: F8, key (rid, item_no).
- **claim (rid, claim_date, claim_type, status, applicant_id, …, remarks)**: everything that depends on rid.

Step 3: third normal form

claim still has transitive dependencies through non-key attributes: rid → applicant_id → name, email, bank_account_no, dept_id (F2), then dept_id → dept_name, location (F5), and rid → approved_by → approval_limit (F7). Neither applicant_id, dept_id nor approved_by is a superkey of claim, and the attributes they determine are not prime, so claim violates 3NF. Decompose:

- **employee (applicant_id, name, email, bank_account_no, dept_id)** from F2; then **department (dept_id, dept_name, location)** from F5.
- **manager (approved_by, approval_limit)** from F7.
- **reimbursement (rid, claim_date, claim_type, status, applicant_id, approved_by, approval_date, remarks)**: what remains of claim.

The same result from the 3NF synthesis algorithm

Applying the textbook's 3NF synthesis algorithm (§7.5.2) to F: one schema per FD of the canonical cover gives R1 = F1, R2 = F2, R3 = (email, applicant_id), R4 = (bank_account_no, applicant_id), R5 = F5, R6 = (dept_name, dept_id), R7 = F7, R8 = F8. R8 contains the candidate key (rid, item_no), so no extra schema is needed. R3 and R4 are contained in R2, and R6 in R5, so they are removed. The result is R1 = reimbursement, R2 = employee (with bank_account_no), R5 = department, R7 = manager and R8 = expense_item: the same tables as Steps 1 to 3.

The final design moves bank_account_no from employee into applicant. That is not a normalization step: it comes from the specialization (A11), because only applicants have a payout account. Both placements are in BCNF.

10.4 The decomposition is lossless and dependency-preserving

- **Lossless (§7.2.3):** at every split, the common attributes are a key of one of the two parts. For example, claim ∩ expense_item = {rid}, which is the key of claim; claim ∩ employee = {applicant_id}, the key of employee; and similarly for department and manager. By the textbook's test, each binary decomposition is lossless, so the whole sequence is.
- **Dependency-preserving (§7.4.4):** every FD F1 to F8 has all its attributes inside one final table (F3 and F4 in employee/applicant, F6 in department), so each can be checked inside one table, by its PRIMARY KEY or UNIQUE constraint. No FD needs a join to check.
- **Agreement with the ER design (§7.9.1):** normalization produced exactly the tables obtained from the ER diagram. This is what the textbook expects from a careful ER design, and it is good evidence that the diagram is right.

11 Integrity constraints

Every rule the data must obey, where it is enforced and why. C = declared in the table (key, NOT NULL or CHECK). T = a rule that needs the old row, another row or another table, so it cannot be a CHECK in MySQL; the application checks it in the form and again in the API (K1). A = other application rules.

11.1 Constraint catalogue

ID	Rule	Mechanism	Reason
C1	Primary keys	PRIMARY KEY	Every table: entity identity (§6.5, §4.4).
C2	Candidate keys: dept_name, email, bank_account_no	UNIQUE	Natural keys stay unique although they are not primary keys (A1).
C3	Foreign keys (10), with the delete rules of Section 11.2	FOREIGN KEY	Referential integrity for every relationship and subclass (§4.4.5).
C4	Mandatory attributes: every column marked "no" in Section 8.2	NOT NULL	Includes dept_id and applicant_id, which enforce total participation in WORKS_IN and SUBMITS.

ID	Rule	Mechanism	Reason
C5	status IN (SUBMITTED, APPROVED, REJECTED, PAID)	CHECK	Approved change 3.
C6	claim_type IN (TRAVEL, MEDICAL, BUSINESS)	CHECK	Approved change 2.
C7	category IN (FOOD, LODGING, FARE, FUEL, LOCAL_ TRANSPORT, MEDICAL, SUPPLIES, OTHER)	CHECK	Approved change 6.
C8	status = SUBMITTED ⇔ approved_by IS NULL and approval_ date IS NULL	CHECK	A decision exists exactly when the claim has left SUBMITTED.
C9	status = REJECTED ⇒ remarks IS NOT NULL	CHECK	A rejection needs a reason (approved change 1).
C10	approved_by <> applicant_id	CHECK	No one approves their own claim (original diagram note).
C11	approval_date >= claim_date	CHECK	A claim cannot be decided before it is filed.
C12	amount > 0, approval_limit > 0, distance_km > 0	CHECK	Zero or negative values are meaningless.
C13	employee_phone.phone matches + and digits, 7 to 15 characters	CHECK	Basic format check.
C14	email matches the pattern name@domain.tld	CHECK	Basic format check; the login depends on it.
T1	Claim workflow: (a) a new claim is always inserted as SUBMITTED, and only the moves in the status diagram are allowed; (b) applicant_id, claim_type and claim_date never change, and a decision, once made, is never edited; (c) on approval the claim has at least one item and SUM(amount) ≤ the approver's approval_limit	Application (form + API)	Needs the old row (a, b) or other tables (c), so a CHECK cannot do it. The API locks the claim and the manager row (FOR UPDATE) while it checks and saves the decision. There is no endpoint that edits a saved claim.
T2	Expense items: expense_date ≤ claim_date, and items cannot change once saved	Application (form + API)	claim_date is in another table. The API offers no way to edit or delete a saved item, so the total a manager approved cannot change afterwards.
T3	The details row goes into the table that matches claim_type	Application (API)	Disjointness of the claim specialization (S2). The claim and its one details row are inserted in the same transaction (A1).
T4	supervisor_id is not the employee and not one of their subordinates, at any depth	Application (form + API)	No supervision cycles: needs a walk up the reporting tree. The form disables supervisors that would create a cycle; the API locks the employee rows and checks again.
A1	A claim, its subclass row and at least one item are inserted in one transaction	Application	Totality of S2 and of CONTAINS (K2).
A2	claim_date is set by the server, never taken from the form	Application	MySQL does not allow CURRENT_ DATE inside a CHECK.
A3	Passwords are stored only as bcrypt hashes	Application	A4.

11.2 What happens on delete

Foreign key	References	ON DELETE	Reason
employee_phone.employee_id	employee	CASCADE	A phone number is part of the employee.
applicant.employee_id, manager.employee_id	employee	CASCADE	A subclass row is the same entity as its superclass row (§6.8.4: delete from all lower-level sets).

Foreign key	References	ON DELETE	Reason
employee.dept_id	department	RESTRICT	Move the employees before deleting their department.
employee.supervisor_id	employee	RESTRICT	Reassign subordinates first. T4 checks this column, and keeping it RESTRICT avoids silent changes to the reporting tree.
reimbursement.applicant_id	applicant	RESTRICT	Claims are financial records and are kept. In effect, an employee with claims cannot be deleted.
reimbursement.approved_by	manager	RESTRICT	The record of who decided must survive. Also required: the column is used in CHECK C10.
travel_/medical_/business_ reimbursement.reimbursement_id	reimbursement	CASCADE	Subclass row is part of the claim.
expense_item.reimbursement_id	reimbursement	CASCADE	Weak entity: existence depends on the claim (W1).

ON UPDATE is the default (RESTRICT) everywhere: primary keys are surrogates and never change (A1).

11.3 Claim status workflow

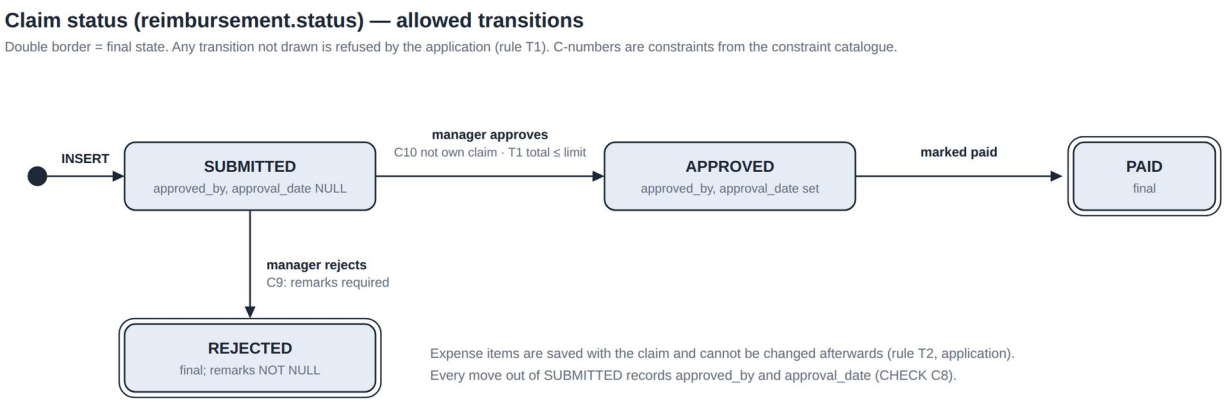


Figure 3. Allowed status changes. The application (rule T1) refuses every other change, for example APPROVED → SUBMITTED or REJECTED → PAID.

11.4 Constraint decisions

K1 CHECK for rules on one row; the application checks rules across rows or tables

Decision	Single-row rules are CHECK constraints (C5 to C14). Rules that need the old row, another row or another table (T1 to T4) are checked by the application, twice: in the browser form, so the user sees the problem before submitting, and again in the API, inside the transaction that writes the data. Both use the same code (shared/validation.mjs). The database has no triggers.
Why	A CHECK is declarative and cannot be bypassed, so every rule that fits in one row is a CHECK. Standard SQL allows subqueries in CHECK, but MySQL does not, and it has no assertions (§4.4.8), so T1 to T4 cannot be declared in the schema. The database-side alternative is triggers (§5.3). They were not part of our course, so we keep these rules in application code that we wrote and can explain. One shared module means the form and the API can never disagree about a rule.
Concurrency	The API reads the rows a rule depends on with SELECT … FOR UPDATE inside the transaction (§4.3, §18.8.3). Two managers deciding the same claim at the same moment therefore cannot both succeed, and two supervisor changes cannot together create a cycle.

Cost	Only the application enforces T1 to T4. A statement typed directly into MySQL could break them, so data is changed only through the application; the application's database user (03_app_user.sql) has no rights to change the schema.
Requirement	MySQL 8.0.16 or later: earlier versions parse CHECK constraints but ignore them.

K2 Two totality rules are enforced by one transaction

Decision	The application inserts a claim, its subclass row and its items in a single transaction (rule A1 in Section 11).
Why	A rule such as "every claim has at least one item" can only be checked after all rows are inserted. That needs a deferred constraint or an assertion (§4.4.7), and MySQL has neither. Rule T1 is the safety net: the application never approves a claim with no items.

K3 Delete behaviour follows the meaning of each reference

Decision	CASCADE only where the child is part of the parent: phones, role rows, claim subclass rows and expense items. RESTRICT everywhere else.
Why	Claims are financial records. An employee who has claims cannot be deleted; RESTRICT on reimbursement.applicant_id stops it. A supervisor or department cannot be deleted while people still point at it; they must be reassigned first. MySQL also forbids ON DELETE actions on columns used in a CHECK, so applicant_id and approved_by (both in C10) must be RESTRICT anyway.

K4 Fixed value lists are CHECK ... IN, not lookup tables or ENUM

Decision	status, claim_type and category are VARCHAR columns with a CHECK ... IN (...) constraint.
Why	A lookup table pays off when the values have attributes of their own or change at run time. These lists are short, fixed, and part of the program's logic (the workflow depends on the exact status values). CHECK is standard SQL (§4.4.4). MySQL's ENUM is non-standard and sorts by position rather than by name.
Cost	Adding a category is a schema change (ALTER TABLE), which is acceptable for a list that rarely changes.

K5 Money is DECIMAL, never FLOAT

Decision	amount DECIMAL(10,2) and approval_limit DECIMAL(12,2).
Why	FLOAT is binary floating point and cannot store 0.10 exactly, so sums of rupee amounts drift. DECIMAL is exact (§4.5).

12 Open questions for the team

Each question lists the assumption that this document, the SQL scripts and the application implement, and what changes if the team decides otherwise.

#	Question	Assumed in this design	Change if the answer differs
Q1	Must the approver be the applicant's supervisor? (The plan marks it settled but does not record the answer.)	No; any other manager within their limit (R6)	One extra check in T1; no table changes
Q2	Do we record who marked a claim paid, and the payment date or reference?	No; only status = PAID (E4)	Add ADMIN subclass and PAYS relationship (paid_by, payment_ date, payment_ref)
Q3	What happens to a claim larger than every manager's limit? Two-level approval?	It stays SUBMITTED until a manager with a high enough limit decides it	Many-to-many APPROVES with a separate approval table
Q4	Store the IFSC code with the bank account?	No; account number only, as in the plan (A11)	Add ifsc_code; candidate key becomes (ifsc_code, bank_ account_no)
Q5	May an applicant withdraw (delete) a claim while it is still SUBMITTED?	No; claims are never deleted by the app	A delete endpoint that allows it only while SUBMITTED
Q6	Is a receipt mandatory for every expense item?	No; receipt_path may be NULL (A7)	Make receipt_path NOT NULL

Appendix A Assignment checklist

The assignment (references/DBMS Project.docx) asks for a report with seven items. All seven are covered here. File 1 holds the source code and the screenshots of the forms.

Required in the report	Where	Status
1. Problem statement	Section 2	Done
2. Objectives	Section 3	Done
3. Functional requirements	Section 4	Done
4. ER diagram	Section 6, Figure 1 (every arrow and line listed in 6.2 and 6.3 and justified in 6.4 to 6.7)	Done
5. Relational schemas obtained from the ER diagram	Sections 7 and 8, Figure 2	Done
6. Functional dependencies that must hold on each table	Section 9	Done
7. Each table normalized up to 3NF	Section 10 (all tables are in BCNF)	Done
File 1: source code and form screenshots	db/, server/, client/, shared/; 15 screenshots in report/screenshots/	Done

Appendix B Questions to expect in the viva

Why is APPROVES not a table?

It is many-to-one from CLAIM_PROCESS to MANAGER, so it folds into the table of the "many" side, which is reimbursement (§6.7.6). Partial participation means the columns are nullable. Folding also lets C8 to C11 be single-row CHECKs (M2).

Why aggregation and not a ternary relationship?

A ternary relationship would store the applicant twice, in SUBMITS and in APPROVES, with nothing to keep them equal (§6.8.5). And since SUBMITS is many-to-one, the aggregation's key is reimbursement_id, so it costs no extra table (G1).

Is a binary MANAGER–REIMBURSEMENT relationship not the same?

In tables, yes: identical. We keep the aggregation because the decision is about a submission (who claims what); the self-approval rule compares the approver with the submitter (G1).

Why is EXPENSE_ITEM weak?

It cannot be identified without its claim: item_no restarts at 1 in every claim. Its key is (reimbursement_id, item_no), and deleting a claim deletes its items (W1).

Why is the EMPLOYEE specialization overlapping but the claim one disjoint?

A manager can also file claims, so roles combine. A claim has exactly one purpose and one set of type details (S1, S2).

How is "disjoint and total" enforced?

Disjoint: claim_type, and the application writes only the matching details row (T3). Total: one transaction inserts the claim and its subclass row; MySQL has no assertions or deferred constraints to do it in the schema (S2, K2).

Why method 1 for the claim ISA, when method 2 is allowed for disjoint and total?

With method 2, expense_item and approved_by would have no single table to reference (S3).

Where is the transitive dependency?

In the original design: employee → department → location. Making DEPARTMENT an entity set removes it (E1, Section 10.3).

Does (travel_from, travel_to) → distance_km hold?

No: distance is a fact of the trip, not of the city pair. If it held, travel_reimbursement would not be in 3NF and we would split off route (N1).

Why not store total_amount?

It is derived; storing it creates an update anomaly. The view computes it (A3).

Why surrogate keys?

Natural keys (email, name) can change, and primary keys must not. The natural keys are still UNIQUE (A1).

Which rules could not be CHECK constraints, and why?

Status transitions (needs the old row), the approval limit (needs another table), frozen items and type matching (other tables), and supervision cycles (recursive). Those are T1 to T4. MySQL allows no subqueries in CHECK and has no assertions, so the application checks them, in the form and again in the API inside a locked transaction (K1).

Why no triggers?

Triggers were not part of our course. The cross-table rules (T1 to T4) are checked by one shared validation module, used by both the form and the API. The API checks again inside the transaction with SELECT … FOR UPDATE, so concurrent requests cannot slip past a rule (K1).

Is the decomposition lossless?

Yes. At every split the common attributes are a key of one side (§7.2.3). It also preserves dependencies (Section 10.4).